CHAPTER

3

文字列の正規化

画面では同じに見える二つの文字列が、プログラムの中では別物として扱われることがある。検索に引っかからない、重複したはずのデータが二件残る、ファイル名で並べると順序が崩れる。こうした不具合の多くは、Unicode の正規化を知っていれば防げる。

本章では、まず同じ文字に二つの表し方がある理由を見て、Unicode が定める四つの正規化形式を整理する。次に Python の標準ライブラリで実際に変換し、最後に、検索キーを作るときに正規化で失われる情報について述べる。

3.1 同じに見えて違う文字列

「が」という文字は、Unicode では二通りに表せる。一つは「が」そのものに割り当てられた符号位置 U+304C を使う方法、もう一つは「か」（U+304B）の後ろに結合用の濁点（U+3099）を置く方法である。前者を合成済み文字、後者を結合文字列と呼ぶ。どちらも画面には同じ「が」として表示されるが、符号位置の並びが違うので、単純な比較では等しくならない。UTF-8 で書き出すと、前者は 3 バイト、後者は 6 バイトになる（図3-1）。

この違いが表に出やすいのはファイル名である。macOS の以前のファイルシ

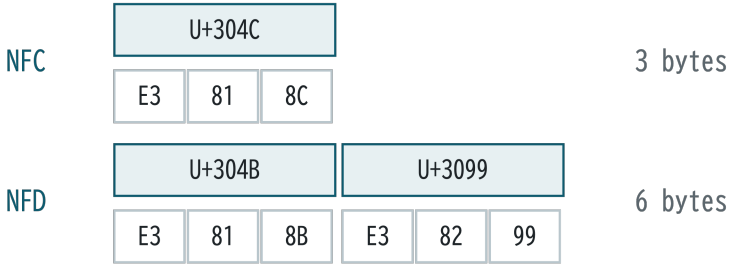


図 3-1　「が」の二つの表し方。上段が符号位置、下段が UTF-8 のバイト列

ステム HFS+ は、ファイル名を分解した形で保存していた[1]。Mac で作った「データ.csv」を Windows や Linux のサーバーにコピーすると、「テ」と濁点が分かれた名前のまま届く。見た目は同じ名前のファイルが二つ並んだり、プログラムからファイルを開けなかったりするのはこのためだ。

3.2　四つの正規化形式

Unicode は、こうした表し方の違いをそろえる手順を「正規化形式」として定めている[2]。正規化形式は、二つの観点の組み合わせで四つある。

一つ目の観点は、合成するか分解するかである。分解（decomposition）は「が」を「か」と濁点に分け、合成（composition）は分けたものを一文字に戻す。二つ目の観点は、どこまでを同じ文字とみなすかである。正準等価は、見た目も意味も同じものだけを同一視する。互換等価は、半角カナと全角カナ、丸数字と数字のように、形は違っても同じ文字として扱えるものまで同一視する。

---

[1] HFS+ が使うのは、Unicode の規格の NFD に近い独自の分解形である。2017 年に導入された APFS は、ファイル名を書かれたとおりに保存する。

[2] Unicode の版ごとに改訂される標準附属書、Unicode Standard Annex #15「Unicode Normalization Forms」（unicode.org/reports/tr15）。

- NFC：正準分解したあと、正準合成する
- NFD：正準分解する
- NFKC：互換分解したあと、正準合成する
- NFKD：互換分解する

日本語の文字がそれぞれの形式でどう変わるかを表3-1に示す。NFC と NFD が変えるのは濁点と半濁点の付け方だけで、文字の種類は変わらない。一方、NFKC と NFKD は、半角カナを全角に、全角英数字を半角に、「①」を「1」に、「㍼」を「平成」に置き換える。

入力	NFC	NFD	NFKC
が（U+304C）	U+304C	U+304B U+3099	U+304C
ｶﾞ（U+FF76 U+FF9E）	そのまま	そのまま	ガ（U+30AC）
ＡＢＣ（全角）	そのまま	そのまま	ABC
①	そのまま	そのまま	1
㍼	そのまま	そのまま	平成

表 3-1　日本語の文字と四つの正規化形式（NFKD は、NFKC で合成された文字を分解した形になる）

3.3　Python で正規化する

Python では、標準ライブラリの `unicodedata` モジュールにある `normalize()` 関数で正規化できる。第 1 引数に形式の名前を、第 2 引数に文字列を渡す。形式の名前は「NFC」「NFD」「NFKC」「NFKD」のいずれかで、それ以外を渡すと `ValueError` になる。

```
normalize.py
import unicodedata

s = "ｶﾞｲﾄﾞ ＡＢＣ ①"
for form in ("NFC", "NFKC"):
    print(form, unicodedata.normalize(form, s))
```

実行すると、NFC の行には入力がそのまま出力され、NFKC の行には「ガイド ABC 1」が出力される。半角の「ｶ」と「ﾞ」が、一文字の「ガ」になっている点に注意してほしい。互換分解で「ｶﾞ」が「カ」と結合用の濁点に分かれ、続く正準合成で「ガ」にまとめられるからだ。

Python 3.8 で加わった `is_normalized()` 関数を使うと、文字列がすでに正規化されているかどうかを調べられる。読み込んだデータのうち変換の要るものだけを選べるので、大量のファイル名やレコードを処理するときに役に立つ。

3.4　検索キーを作るときの注意

利用者が入力した語で検索する場合、検索キーと検索対象の両方を NFKC で正規化しておけば、半角と全角の違いを気にせずに照合できる。ただし、NFKC は情報を捨てる変換でもある。「①」は「1」に、「㈱」は「(株)」になり、元の文字には戻せない。

> **ポイント**
>
> 正規化した文字列は、検索と照合のためだけに使う。画面に表示する文字列と保存する文字列には、利用者が入力したものをそのまま残しておく。

もう一つ、NFKC でもそろわない揺れがある。波ダッシュ「〜」（U+301C）と全角チルダ「～」（U+FF5E）だ。NFKC は全角チルダを半角の「~」に変える

が、波ダッシュは変えない[1]。そのため「10〜20」と「10〜20」は、NFKC を通しても一致しない。こうした文字は、正規化とは別に対応表を用意して置き換える必要がある。

Python なら、正規化した文字列に、`str.maketrans()` で作った対応表を `translate()` で当てればよい。NFKC が全角チルダを「~」に変えるので、表には波ダッシュだけを書けば足りる。

search_key.py

```
import unicodedata

WAVE = str.maketrans("〜", "~")

def search_key(s):
    return unicodedata.normalize("NFKC", s).translate(WAVE)
```

こうすれば、「10〜20」と「10〜20」は、どちらも「10~20」という同じキーになる。

A specimen chapter written for the Postext Cookbook; the book and its other chapters are fictitious. Set in Noto Serif JP, Noto Sans JP and BIZ UDGothic (SIL OFL). Text: CC BY 4.0.

[1] Shift_JIS の 0x8160 は、JIS の対応表では波ダッシュに、マイクロソフトの CP932 の表では全角チルダに変換される。同じ文書でも、変換に使った表によって符号位置が分かれる。